

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Reverse Engineering

Code of Conduct:

I G. GOWTHAMI(192524285) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:G. GOWTHAMI

Annexure A

Reverse Engineering

1. Reverse Engineering a Recommendation System

A large-scale e-commerce platform provides highly accurate product recommendations using real-time data. Analyze the system and reverse engineer its architecture using PySpark and RDD operations. Identify how data is collected, processed, and transformed, and infer the role of intelligent agents in updating recommendations dynamically.

2. Reverse Engineering a Smart Traffic Management System

A city-wide traffic system adapts signals in real time based on vehicle density and congestion patterns. Reverse engineer the underlying AI system by describing the data pipeline using RDD-based streaming, identifying agent interactions, and explaining how decisions are made under time constraints.

3. Reverse Engineering a Fraud Detection Pipeline

A banking system flags fraudulent transactions within milliseconds. By observing its outputs and behavior, reconstruct the distributed AI pipeline using PySpark RDDs. Explain how feature extraction, anomaly detection, and agent-based alert mechanisms are likely implemented.

4. Reverse Engineering a Personalized Learning System

An online learning platform adapts content difficulty based on student performance and engagement. Reverse engineer the system design by identifying how learner data is processed using RDD transformations, and explain how intelligent agents contribute to adaptive learning and feedback mechanisms.

5. Reverse Engineering a Cloud-Based AI Monitoring System

A cloud platform automatically scales resources and detects system anomalies in real time. Reverse engineer the architecture by analyzing how PySpark processes large-scale logs, how RDD operations support fault tolerance, and how intelligent agents coordinate monitoring, scaling, and anomaly response.

ANSWERS

1. Reverse Engineering a Recommendation System

An e-commerce recommendation system collects user clicks, searches, purchases, ratings, and browsing history. PySpark RDDs process this data using operations such as `map()`, `filter()`, `groupByKey()`, and `reduceByKey()`.

```
data = sc.textFile("user_events")
```

```
purchases = data.filter(lambda x: "purchase" in x)
```

Intelligent agents analyze user behavior, rank products, and continuously update recommendations based on user feedback.

Flow: User Data → RDD Processing → Recommendation Model → Intelligent Agent → Product Recommendation.

2. Reverse Engineering a Smart Traffic Management System

Traffic systems collect vehicle density, speed, congestion, and signal data from sensors and cameras. Spark processes this data using RDD-based streaming.

```
traffic = stream.map(lambda x: x.split(","))  
congestion = traffic.filter(lambda x: int(x[2]) > 100)
```

Traffic agents analyze congestion and automatically adjust signal timings. Emergency agents can give priority to ambulances and other emergency vehicles.

Flow: Sensors → RDD Processing → Congestion Detection → Agents → Signal Control.

3. Reverse Engineering a Fraud Detection Pipeline

Banks collect transaction amount, location, time, merchant, and customer history. PySpark RDDs transform transactions and extract useful features.

```
data = sc.textFile("transactions")  
high_value = data.filter(lambda x: float(x.split(",")[2]) > 10000)
```

A fraud detection agent calculates risk and identifies unusual transactions. An alert agent immediately informs the bank or blocks suspicious activity.

Flow: Transaction → Feature Extraction → Anomaly Detection → Risk Agent → Alert.

4. Reverse Engineering a Personalized Learning System

An online learning platform collects quiz scores, attempts, time spent, and student engagement. RDDs aggregate this information to measure performance.

```
scores = data.map(lambda x: (x[0], float(x[3])))
```

```
performance = scores.reduceByKey(lambda a, b: a + b)
```

Intelligent agents identify weak areas and select suitable content. Difficulty is increased for high-performing students and reduced for struggling students.

Flow: Student Data → RDD Processing → Performance Analysis → Learning Agent → Personalized Content.

5. Reverse Engineering a Cloud-Based AI Monitoring System

A cloud monitoring system collects CPU, memory, network, and application logs. PySpark RDDs process large amounts of monitoring data.

```
logs = sc.textFile("cloud_logs")
```

```
errors = logs.filter(lambda x: "ERROR" in x)
```

Monitoring agents detect anomalies, while scaling agents automatically add or remove cloud resources based on workload.

Flow: Cloud Logs → RDD Processing → Anomaly Detection → Agents → Scaling/Recovery.

Common Architecture

Data Collection → PySpark RDD Processing → AI Analysis → Intelligent Agents → Decision → Action → Feedback

RDDs provide **parallel processing and fault tolerance**, while intelligent agents provide **autonomous decision-making and real-time adaptation**.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution

Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation